

CryptoChain Analyzer Dashboard

Final Report - Hash Functions and Blockchain

Cryptography | Universidad Alfonso X el Sabio | Prof. Jorge Calvo | 2025-26

Author: danyel430 | Deadline: 14 May 2026

1. Cryptographic Metrics Displayed and Their Meaning

M1 - Proof of Work Monitor

The dashboard fetches live block data from the Blockstream API and displays three core Proof-of-Work metrics. Network difficulty quantifies how hard it is to find a valid block hash. Bitcoin adjusts difficulty every 2,016 blocks (~2 weeks) so that the average inter-block time stays close to 600 seconds. The compact bits field in the block header encodes the 256-bit target threshold T :

$$T = \text{coefficient} \times 2^{(8 \times (\text{exponent} - 3))} \quad \text{where bits} = [\text{exponent} \mid \text{coefficient}]$$

A valid block requires $\text{SHA-256}(\text{SHA-256}(\text{header})) < T$, meaning the hash must begin with a certain number of leading zero bits. The dashboard visualises this as a binary bar. The estimated hashrate is derived as $H = \text{difficulty} \times 2^{32} / 600$ (expressed in EH/s). The inter-block time distribution is plotted as a histogram and compared to the theoretical Exponential($\lambda = 1/600$) baseline from Poisson process theory.

M2 - Block Header Analyzer

Displays the full 80-byte block header (version, previous hash, Merkle root, timestamp, bits, nonce) and performs a local Proof-of-Work verification using hashlib:

```
block_hash = SHA256( SHA256( header_bytes ) ) [little-endian byte order]
```

The computed hash is compared against the target derived from the bits field to confirm it is below the threshold. Leading zero bits are counted and displayed. This directly links raw block data to Topic 7 theory: the nonce is the only field miners iterate over; a valid nonce produces a hash below T .

M3 - Difficulty History

A time-series chart shows difficulty across the last ~30 adjustment epochs (each epoch = 2,016 blocks). Each adjustment event is marked. The ratio of actual average block time to the 600-second target is plotted per period, illustrating the feedback-control mechanism from the Nakamoto whitepaper (Section 4): if the previous period was faster than 10 min, difficulty rises; if slower, it falls.

M5 - Merkle Proof Verifier

The user selects any transaction in a recent block. The module downloads all transaction IDs, reconstructs the full Merkle tree layer by layer using double-SHA-256 at each node, and verifies that the computed root matches the merkle_root field in the block header. It then displays the proof path ($O(\log N)$ sibling hashes) needed to recompute the root from a single leaf, making the logarithmic efficiency of Merkle proofs tangible.

M6 - Security Score (51% Attack)

Based on Nakamoto (2008, Section 11), this module models the probability that an attacker controlling fraction q of the hashrate can rewrite the last z confirmed blocks. Attack success follows the

Poisson-binomial approximation:

$$P(z, q) = 1 - \sum_{k=0}^z \left[\text{Poisson}(k, \text{lam}=z \cdot q/p) \times (1 - (q/p)^{(z-k)}) \right]$$

The module also estimates the real-time cost of a 51% attack in USD/hour based on the live network hashrate, ASIC efficiency (Antminer S21 Pro: 234 TH/s at 3,500 W), and an industrial electricity price of \$0.05/kWh. Interactive sliders let the user explore how confirmation depth reduces risk.

2. AI Components: Models, Justification, and Evaluation

M4 - Anomaly Detector (Unsupervised)

Model chosen: Statistical exponential-distribution anomaly detector.

Justification: Bitcoin mining is a Poisson process - each hashing attempt is independent with constant success probability - so inter-block times follow $\text{Exp}(\lambda = 1/600 \text{ s})$ under normal conditions (Nakamoto 2008, Section 11). No labelled dataset of anomalous blocks exists, making unsupervised detection the natural choice.

Method:

- Fit $\text{Exp}(\lambda_{\text{hat}})$ to the last 200 inter-block times via MLE: $\lambda_{\text{hat}} = 1 / x_{\text{mean}}$.
- Compute a two-sided p-value for each observation: $p = 2 \times \min(\text{CDF}(t), 1 - \text{CDF}(t))$.
- Flag as anomaly if $p < \alpha = 0.01$ (bilateral, 0.5% per tail).

Evaluation metrics:

Metric	Description
KS statistic D & p-value	Kolmogorov-Smirnov test of $\text{Exp}(\lambda_{\text{hat}})$ goodness-of-fit. H_0 not rejected ($p > 0.05$) confirms the exponential model is appropriate.
MAE vs $\mu = 600 \text{ s}$	Mean absolute error between sample mean and theoretical target. Typically $< 30 \text{ s}$, confirming the network runs close to its target.
Anomaly rate	Percentage of flagged blocks. Expected $\sim 1\%$ under H_0 ; higher values indicate hashrate instability or pool behaviour.

Limitations: The model assumes constant λ over the ~ 1.4 -day sample window. In practice hashrate fluctuates. A sliding-window approach (CUSUM or PELT) would detect structural breaks in λ more robustly.

M7 - Difficulty Predictor (Supervised) - Second AI Approach

Model chosen: Ridge Regression (L2-regularised linear regression) with sliding-window features on the log-difficulty series.

Justification: Bitcoin difficulty has grown roughly log-linearly since 2013, with approximately 50 biweekly data points available from the Blockchain.info API. At this scale, complex sequence models (LSTM, Prophet) would overfit severely. Ridge Regression on a logarithmic scale is interpretable, regularised, and well-matched to the data size. It also contrasts clearly with M4: block-level unsupervised vs epoch-level supervised.

Feature engineering: a sliding window of $k = 4$ previous difficulty values plus their pairwise percentage changes forms each feature vector. StandardScaler normalisation is applied before fitting. Training uses a temporal split (80% train / 20% test, no shuffling) to preserve time ordering.

Evaluation metrics:

Metric	Meaning
MAE	Mean Absolute Error in difficulty units.
RMSE	Root Mean Square Error - penalises large errors more than MAE.
MAPE	Mean Absolute Percentage Error - scale-independent; typical values 3-8%.
R2	Coefficient of determination in log-space. Values > 0.85 indicate the model captures the dominant trend.

Comparison M4 vs M7:

Criterion	M4 - Anomaly Detector	M7 - Difficulty Predictor
Type	Unsupervised	Supervised
Granularity	Per block (~10 min)	Per epoch (~2 weeks)
Target	Is this block anomalous?	What will next difficulty be?
Core model	Exponential distribution (MLE)	Ridge Regression (L2)
Evaluation	KS test, anomaly rate, MAE	MAE, RMSE, MAPE, R2
Limitation	Assumes constant hashrate	~50 training points

The two models are complementary: M4 monitors individual block behaviour in real time, while M7 anticipates the next network-wide difficulty adjustment.

3. References

- [1] Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. <https://bitcoin.org/bitcoin.pdf> - Core reference for PoW, Merkle trees, difficulty adjustment, and the 51% attack model (Sections 4, 6, 11).
- [2] Blockstream Explorer API. REST API documentation. <https://github.com/Blockstream/esplora/blob/master/API.md> - Used for live block headers, transactions, and hashrate data (M1, M2, M3, M5).
- [3] Blockchain.info API. Charts and statistics endpoints. https://www.blockchain.com/explorer/api/blockchain_api - Historical difficulty series used in M3 and M7.
- [4] Rosenfeld, M. (2011). Analysis of Bitcoin Pooled Mining Reward Systems. arXiv:1112.4980 - Background on mining pool behaviour and selfish mining, motivating the M4 anomaly detector.
- [5] Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825-2830 - Library used for Ridge Regression, StandardScaler, and evaluation metrics in M7.